a take in rca inference

formal mathematical modelling and implementation of this particular alert space and all the stuff related to the grouping that makes it boring to implement

thanh et. nil

June 12, 2026

1. defining the global telemetry tuple space

before we can perform root cause analysis (rca), we must formalize the environment. production telemetry resides in an infinite, high-entropy tuple space.

let $\mathcal{I}$ be incident identifiers, $\mathcal{S}$ the topology of services, $\mathcal{M}$ the metrics, Σ the severity levels, $\mathcal{T}$ the temporal continuum, and $\mathcal{W}^*$ the kleene closure of raw log tokens.

an incident state is defined as an element in the alert-incident tuple space:

$$\mathcal{A}_{\text{tuple}} = \mathcal{I} \times \mathcal{S} \times \mathcal{M} \times \Sigma \times \mathcal{T} \times \mathcal{W}^* \times (\mathbb{R} \times \mathbb{R})$$

this space is continuously expanding, making raw heuristic inference mathematically unstable.

2. the dataset reality: the e0x.json subset

when examining our actual operational datasets, such as the `e0x.json` files, we do not operate on the entire infinite space.

instead, the dataset represents a finite, bounded subset $E \subset \mathcal{A}_{\text{tuple}}$. this subset is restricted to a specific temporal window Δt and a local topological subgraph G_{local} triggered by an initial alert.

however, the problem of rca inference remains the exact same: even within this finite subset E, the telemetry is fragmented, unstructured, and highly variant. the "boring" problem of grouping identical failures that look slightly different (due to timestamps or ip addresses) must be solved algebraically.

3. mathematical preliminary: the equivalence relation

to solve the grouping problem algebraically, we must formalize what it means for two telemetry logs to be "identical" despite ambient noise.

let S be a set. a binary relation $\sim$ on S is an **equivalence relation** if it satisfies three properties for all $a, b, c \in S$:

- **reflexivity:** $a \sim a$
- **symmetry:** if $a \sim b$, then $b \sim a$
- **transitivity:** if $a \sim b$ and $b \sim c$, then $a \sim c$

if a relation satisfies these properties, it partitions the set S into disjoint **equivalence classes**, forming the quotient set $S/\sim$. this is how we mathematically collapse infinite log variations into a finite set of structural patterns.

4. solving the grouping problem: quotient reduction

applying this to our subset E, we define a masking projection $\phi : \mathcal{W}^* \to \mathcal{W}^*_{\text{canonical}}$ to strip out ambient noise.

this projection induces our strict equivalence relation over the log space:

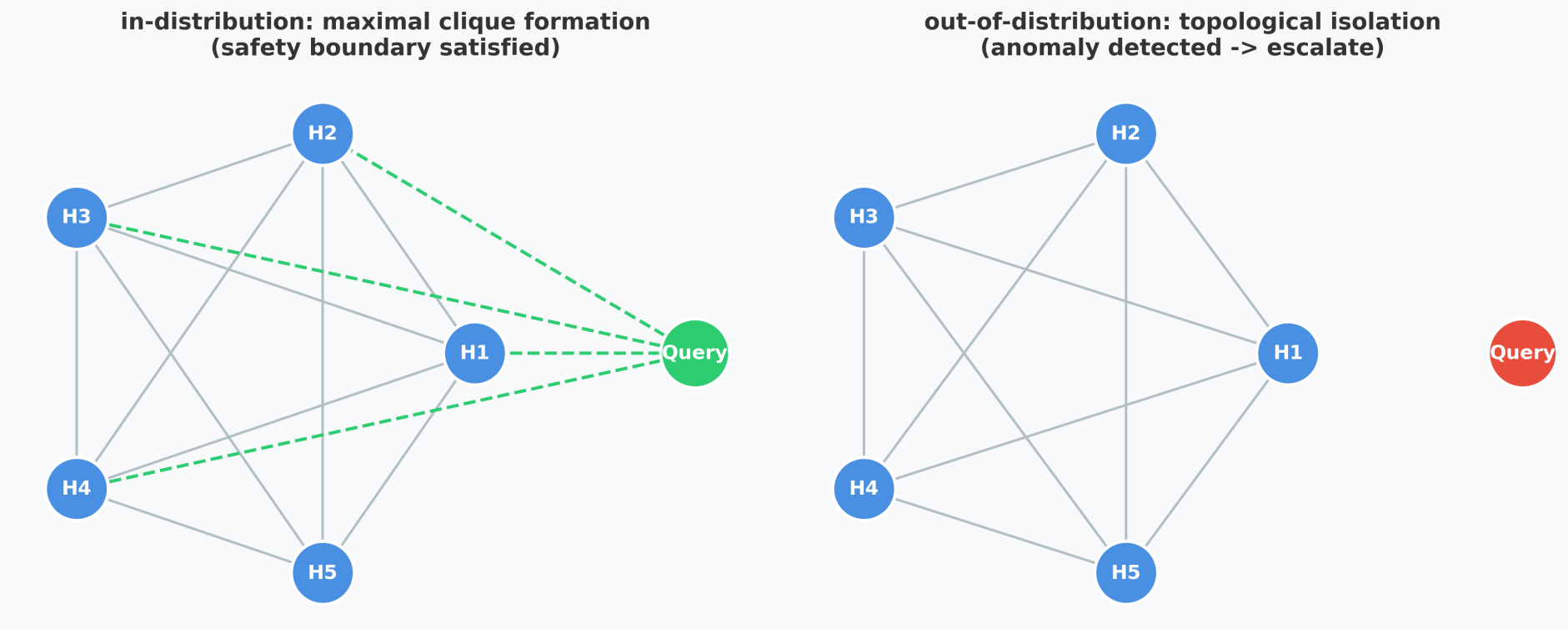
$$w_1 \sim w_2 \iff \phi(w_1) = \phi(w_2)$$

instead of comparing raw strings, the system groups failures by computing strictly over the algebraic quotient set $\mathcal{W}^*/\sim$.

to prevent unrelated failures from colliding in this reduced space, an injective anchor mapping $\alpha([w])$ explicitly preserves critical invariant states, projecting them into immutable tokens like ω_{oom} or ω_{deadlock}.

5. the clique-partitioning problem for rca inference

we map the telemetry into a compatibility graph $G = (V, E)$ where edges require Sim $\geq \epsilon$. this visualizes our out-of-distribution (ood) safety boundary.



if our current `e0x.json` query forms a maximal cohesive clique ($\omega(G_q) \geq k$) with historical incidents (left), inference is mathematically safe. if it is topologically isolated (right), it is ood and the system halts

6. inference to action: expected utility

if the clique bound is satisfied, the system derives an empirical conditional probability distribution $P(\text{RC} \mid I_q)$ from the neighborhood.

rca is meaningless without an optimal remediation path. we map the inferred root cause to a von neumann-morgenstern expected utility maximization over our action catalog $\mathcal{A}_{\text{catalog}}$:

$$\text{EV}(A) = \sum_{c \in \mathcal{RC}} P(\text{RC} = c \mid I_q) \cdot \mathcal{U}(A, c) - [\gamma_1 \cdot \text{cost}_A + \gamma_2 \cdot \text{downtime}_A]$$

$$A^* = \arg \max_{A \in \mathcal{A}_{\text{catalog}}} \text{EV}(A)$$

7. formalizing the orchestration pipeline

to understand the engine's execution flow, we formalize the architecture as a strict mathematical composition of three deterministic mappings:

- $\mathcal{E}$ **(extraction functor):** maps the raw telemetry subset E into the structural quotient space by applying the equivalence relation.

$$\mathcal{E} : E \mapsto \mathcal{W}^* / \sim$$

- $\mathcal{R}$ **(retrieval mapping):** evaluates the quotient space against the clique boundary to generate a conditional probability distribution over root causes.

$$\mathcal{R} : \mathcal{E}(E) \mapsto P(\mathrm{RC} \mid I_q)$$

- $\mathcal{O}$ **(optimization operator):** takes the probability distribution and solves the expected utility supremum to output the optimal infrastructure mutation.

$$\mathcal{O} : P(\mathrm{RC}) \mapsto A^*$$

8. deterministic functional composition

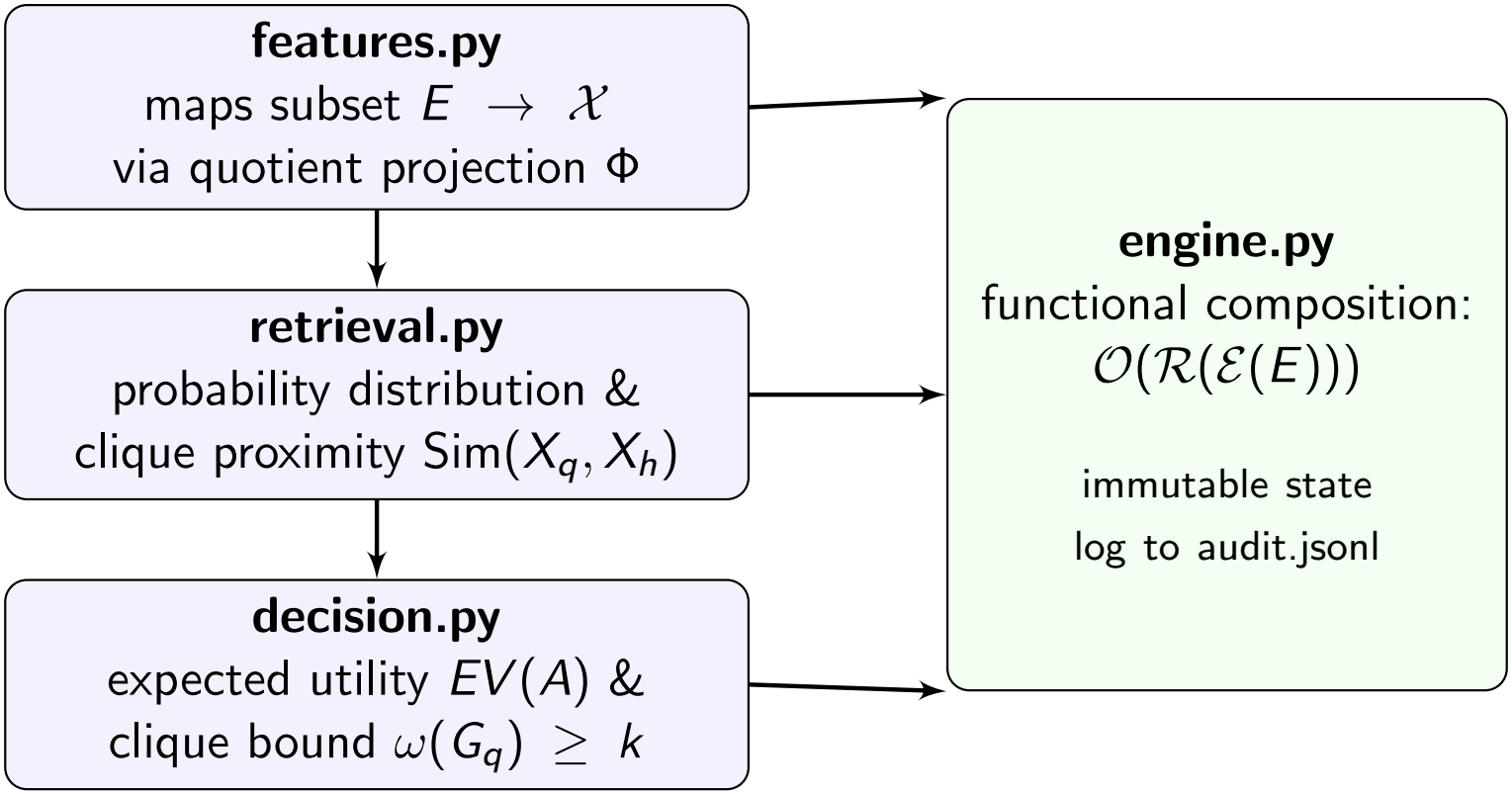
now we come to our formulation. there is no if-else branching, the orchestrator evaluates the incident (observed stochastic process) in functional composition.

$$A^* = \mathcal{O}(\mathcal{R}(\mathcal{E}(E)))$$

why this matters for system reliability:

- **strict idempotency:** identical bounded subsets E will mathematically guarantee the exact same evaluated mutation A^*.
- **mathematical audibility:** every step of the composition is written immutably to `audit.jsonl`. if an incorrect action is triggered, we do not trace convoluted code paths; we simply audit which of the three functions ($\mathcal{E}$, $\mathcal{R}$, or $\mathcal{O}$) failed to bound the variance.

7. architecture diagram: math-to-code mapping



the computer science translation

while the formal specification relies on set theory and decision mathematics, the computational implementation maps directly to standard algorithmic architectures:

- **quotient set reduction $\rightarrow$ canonical tokenization:** the masking projection ϕ is implemented as a deterministic feature extraction pipeline. we use regex to hash high-entropy log chains into canonical semantic tokens, acting as a dimensionality reduction step before retrieval.
- **clique partitioning $\rightarrow$ ϵ-radius k-nearest neighbors (k-nn):** the topological boundary validation is a localized k-nn search. we compute the multi-tier jaccard index across the finite subset. the isolation threshold $\text{Sim} < 0.15$ simply means the query vector falls entirely outside the ϵ-radius of any known historical cluster.
- **expected utility $\rightarrow$ cost-aware greedy heuristic:** the von neumann-morgenstern equation acts as a single-step deterministic

8. applying the math to the codebase

the mathematical reductions are implemented precisely to process the $E \subset \mathcal{A}_{\text{tuple}}$ payload from our `e0x.json` files:

- **features.py (extraction functor):** parses the bounded dataset. regular expressions execute the canonical mapping ϕ to cluster the logs into the quotient space, bypassing the tedious string-matching problem. it then applies the injective anchor mapping α.
- **retrieval.py (proximity engine):** evaluates the localized multi-tier jaccard intersection. it aggregates the historical success paths of the resulting neighborhood $\mathcal{N}_k(X_q)$ to calculate empirical probabilities.

9. executing decisions and orchestration

- **decision.py (boundary and optimization):** tests the topological clique boundary ($\epsilon = 0.15$). if the query subset is isolated, it escalates to human operators. otherwise, it parses the cost matrices from `actions.yaml` and computes the expected utility supremum to find A^*.
- **engine.py (orchestration homomorphism):** handles the deterministic composition of the subset $\mathcal{O}(\mathcal{R}(\mathcal{E}(E)))$. it provides idempotent state logging by serializing the output tensor to `audit.jsonl`.

10. implementing $\mathcal{E}$ and $\mathcal{R}$ in the codebase

the mathematical reductions are implemented precisely to process the bounded data payload from our `e0x.json` files:

- **features.py (the $\mathcal{E}$ functor):** executes the mapping $\mathcal{E} : E \mapsto \mathcal{W}^* / \sim$. it uses a chain of deterministic regular expressions to perform the canonical mapping ϕ. this collapses raw log strings into the structural quotient space, stripping out unique timestamps and hex codes. it then applies the injective mapping α to append semantic anchor tokens.

- **retrieval.py (the $\mathcal{R}$ mapping):** executes the proximity matrix $\mathcal{R} : \mathcal{E}(E) \mapsto P(\text{RC} \mid I_q)$. it evaluates the multi-tier jaccard index between the quotient-reduced query and the historical corpus. it calculates the similarity weights and constructs the neighborhood $\mathcal{N}_k(X_q)$ to output the empirical probability measure.

11. implementing $\mathcal{O}$ and orchestration

- **decision.py (the $\mathcal{O}$ operator):** implements the utility maximization $\mathcal{O} : P(\mathrm{RC}) \mapsto A^*$.
  1. **boundary check:** first, it validates the clique boundary ($\omega(G_q) \geq k$). if the similarity edge $\epsilon < 0.15$, it halts the optimization and forces a human fallback.
  2. **utility optimization:** if validated, it parses the cost penalties from `actions.yaml` and solves the supremum for the expected utility equation to return the remediation action A^*.

- **engine.py (the functional orchestrator):** serves as the entry point that evaluates the full composition $A^* = \mathcal{O}(\mathcal{R}(\mathcal{E}(E)))$. it ensures that every step of the pipeline is idempotent and writes the resulting decision tensor to `audit.jsonl` for absolute mathematical auditability.

10. mathematical limitations: pod restart bias

the expected utility formula exhibits a systemic vulnerability under high uncertainty. let A_r be a pod restart and A_m be a rollback. the penalty thresholds for A_r approach zero:

$$\lim_{\mathbf{C}(A_r)\to 0} [\gamma_1 \cdot \text{cost}_{A_r} + \gamma_2 \cdot \text{downtime}_{A_r}] \approx 0$$

when inference uncertainty is high within a bounded subset E, the probability distribution $P(\text{RC} \mid I_q)$ flattens. the expected utility computation reduces to:

$$\text{EV}(A_r) \approx \bar{\mathcal{U}} - 0 \quad \gg \quad \text{EV}(A_m) \approx \bar{\mathcal{U}} - \text{penalty}$$

therefore, A_r acts as an absorbing state. the engine is mathematically biased toward temporary mitigations when the `e0x.json` data subset lacks strong clique cohesion.